CS231P - Parallel and Distributed Computing

Group 20: Homework 3

Comparative Analysis Between Sequential & Parallel
Implementations of Prime Numbers Counter Program

Omkar Dabir
UCI Net ID: odabir
odabir@uci.edu

Sanket Landge
UCI Net ID: slandge
slandge@uci.edu

Mayuresh Muley
UCI Net ID: muleym
muleym@uci.edu

1. SYSTEM DESCRIPTION

The experiments were conducted on an Apple MacBook Air (Model: Mac16,12) equipped with the Apple M4 chip. The M4 has 10 cores total: 4 high-performance cores and 6 efficiency cores, with 10 logical CPUs. The system has 16 GB of unified memory and runs macOS 26.0.1. The programs were compiled with Apple Clang using the -fopenmp flag and the libomp library installed via Homebrew.

2. IMPLEMENTATION

2.1 Sequential Version

The sequential implementation (primes_seq) iterates over all integers from 0 to max and applies is_prime() to each candidate. The primality test uses two standard shortcuts: divisors are only tested up to the square root of the candidate, and even numbers greater than 2 are eliminated immediately. This reduces the per-number cost from O(n) to O(sqrt(n)) divisions.

2.2 Parallel Version

The parallel implementation (primes_par) uses OpenMP with a shared counter incremented via #pragma omp atomic capture. This is a hardware-level atomic read-increment operation that is cheaper and deadlock-free compared to a mutex. The original mutex-based approach (omp_set_lock / omp_unset_lock) caused the program to hang indefinitely for thread counts above 8 due to severe lock contention and was replaced with the atomic directive. Each thread reads the current counter value and increments it in a single uninterruptible step, then tests that value for primality. This work-stealing scheme achieves dynamic load balancing: threads that finish quickly immediately claim the next integer rather than waiting on a static partition.

3. PERFORMANCE RESULTS

Each configuration was executed three times over the range [1, 10^7] with output suppressed. The table below reports the average time and the speedup relative to the sequential baseline.

Implementation	Threads	Avg Time (s)	Speedup
Sequential	-	0.78011800	1.0000
Parallel	1	0.80632500	0.9675
Parallel	2	0.42099800	1.8530
Parallel	4	0.25619800	3.0450
Parallel	8	0.30299500	2.5749
Parallel	16	0.34682500	2.2493

Implementation	Threads	Avg Time (s)	Speedup
Parallel	32	0.34680200	2.2495
Parallel	64	0.34871500	2.2371
Parallel	128	0.34873900	2.2369

4. ANALYSIS

4.1 Correctness

Correctness was verified by comparing the sorted output of both implementations on the range [1, 100,000]. Both the outputs were identical, confirming that the lock-protected counter prevents any values from being skipped or duplicated.

4.2 Speedup Analysis

The results show clear speedup up to 4 threads, followed by a performance plateau. The sequential baseline ran in 0.780 s, while the parallel version peaked at 4 threads with 0.256 s - a 3.04x speedup. Beyond 4 threads, execution time increased to around 0.303 s at 8 threads and plateaued at approximately 0.347 s from 16 to 128 threads.

The peak at 4 threads aligns exactly with the 4 performance cores of the Apple M4. Beyond 4 threads, the 6 efficiency cores are recruited - these are lower-frequency cores optimized for power efficiency rather than throughput, which explains the slowdown from 4 to 8 threads. The hard plateau from 16 to 128 threads reflects full saturation of all 10 physical cores combined with atomic contention overhead on the shared counter as excess threads spin.

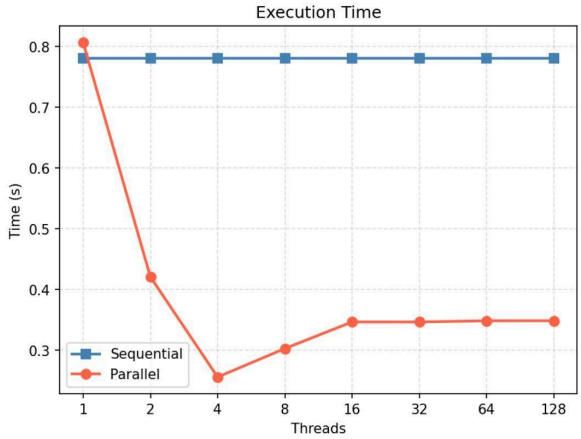

***Figure 1**. Execution time (s) vs. thread count*

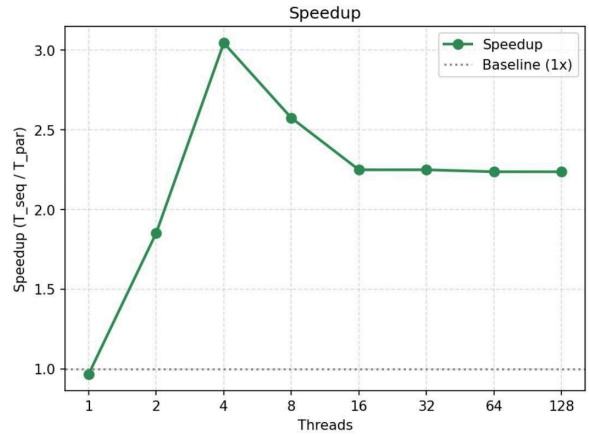

***Figure 2**. Speedup (T_seq / T_par) vs. thread count*

4.3 Sequential vs. Parallel with 1 Thread

The 1-thread parallel version (0.806 s) is approximately 3.4% slower than the sequential version (0.780 s). This overhead comes from three sources:

- OpenMP thread initialization and teardown overhead.
- 10,000,000 atomic read-increment operations per run (#pragma omp atomic capture).
- Additional branching in the while(1) dispatch loop versus a simple for loop.

This demonstrates that synchronization primitives carry a measurable cost even without contention. Parallelization is not free and must be justified by available hardware concurrency.

4.4 Effect of Simultaneous Multi-Threading

The test system has no SMT - each physical core is a single hardware thread, so logical and physical CPU counts are identical (both 10). On x86 processors with SMT (Intel Hyper-Threading), enabling it doubles the logical thread

count and can yield modest speedups for memory-bound workloads. For this compute-bound prime-checking task, SMT would provide limited benefit even on x86, since the bottleneck is integer arithmetic rather than memory access.

5. CONCLUSION

Both implementations correctly identify all primes in [1, 10^7]. The parallel implementation replaces the original mutex-based counter with #pragma omp atomic capture, which eliminates deadlocks that occurred at high thread counts and reduces synchronization overhead. On the Apple M4, the parallel version achieves a peak speedup of 3.04x at 4 threads - matching the 4 performance cores exactly. Beyond 4 threads, efficiency cores and atomic contention cause performance to plateau at approximately 2.25x. These results confirm a key principle: real speedup requires matching the thread count to available hardware parallelism, and the type of synchronization primitive matters significantly at scale.